PLC Structured Text Code Generation and Validation Experimental Report

Final Project Report | November 2026

Abstract

This project builds a dataset and a lightweight agent harness for generating IEC 61131-3 Structured Text (ST). It compares an existing agent with a custom harness, each paired with two foundation models. The dataset contains 20 assigned public problems and 10 original synthetic problems. Each task includes an interface, assumptions, scan-level Linear Temporal Logic (LTL) specifications, and test scenarios.

The four experimental conditions produced 120 archived outputs. Review found that the original CODESYS adapter could report project success without calling the candidate. A uniform offline recheck of the unchanged outputs confirmed 52 passes, found three CODESYS compile failures, and rejected 65 incomplete source files before compilation. The archived 90 project passes and the subsequent 30 project passes are retained for traceability. Total recorded API usage was CNY 6.099742. The recheck made no model requests.

Additional validation includes 20/20 automated STruC++ tests across four P02 outputs and 918 CBMC properties checked on their translated code, with no failures. On CODESYS Control Win V3 x64 version 3.5.22.40, 88 of 224 planned model-scenario executions were run for the custom agent with GPT and DeepSeek: 82 passed and 6 failed, covering 56 of 112 distinct scenario IDs. The other 136 model-scenario executions did not reach runtime. PLCVerif was not run. Conclusions are limited to this dataset, these model versions, and the prompt configuration used.

1 Scope and Completion Status

The assignment requires all public problems and at least 10 self-developed problems, formal specifications and test cases for every problem, a custom ST-generation agent, a comparison with an existing agent using at least two models, and a report of compile, test, verification, and resource results.

This project delivers the dataset, agent harness, CODESYS offline-compile adapter, complete 120-cell matrix, automated ST tests, CODESYS runtime scenario tests, and bounded formal-check evidence. All 30 tasks were run under all four experimental conditions. Total recorded API usage was CNY 6.099742, below the CNY 18 limit. Control Win executed a subset of runtime scenarios for the custom agent with GPT and DeepSeek. P02 also has 20 STruC++ test results and 918 CBMC property checks. The report states the coverage limits and does not present partial execution as full-task success.

Assignment requirement	Project evidence	Status
20 public problems and at least 10 self-developed problems	30 tasks in dataset/tasks.json	Complete
Source, assumptions, LTL, and test scenarios for each task	Dataset fields and docs/DATASET_SPEC.md	Complete
Custom PLC ST agent harness	src/agent_harness.py, model adapters, and CODESYS compiler adapter	Connected to CODESYS; bounded compile-repair loop implemented
Existing agent and two models	Pi coding-agent 1.5.0; DMXAPI models gpt-6.1-sol and deepseek-v4.1-flash-1017	All four conditions run on all 30 tasks
Four-condition experiment matrix	experiments/results.csv	120/120 cells complete; all 30 tasks paired across conditions
Compile, runtime and formal checks	Linked recheck and archived tests	52/120 linked builds passed. Historical runtime: 82/88 executed. P02: 20/20 automated tests and 918 CBMC checks
Cost and resource records	experiments/*usage*.jsonl and results_summary.json	234 proxy records (233 HTTP 200 responses and one budget rejection); total recorded usage CNY 6.099742

2 Experimental Environment

Item	Configuration
Host	Windows
Python	3.12
Existing agent	@earendil-works/pi-coding-agent 1.5.0; file, shell, MCP, extensions, and session features disabled
Pi Node.js	Side-by-side runtime 22.19.0; the system default Node.js was not changed
API	DMXAPI OpenAI-compatible Chat Completions through a cost-recording proxy bound only to 127.0.0.1
Model A	gpt-6.1-sol
Model B	deepseek-v4.1-flash-1017
Target compiler	CODESYS V3.5 SP22 Patch 4 (3.5.22.40), CODESYS Control Win V3 x64, device ID 0000 0004
PLC simulation runtime	CODESYS Control Win V3 x64 3.5.22.40; custom-agent scenarios ran in a downloaded 50 ms cyclic task; no physical I/O was mapped
Formal verification tool	PLCVerif was not detected; no verifier adapter was configured

The cost proxy allowed only the two listed model IDs. It recorded the model, token counts, provider-reported cost, elapsed time, HTTP status, and timestamp. It did not record prompts, response bodies, or API keys. The API key was supplied through the `DMXAPI_API_KEY` process environment variable and was not included in the submitted files.

3 Dataset and Formal Specifications

3.1 Sources and task coverage

The 20 public problems were taken from the assignment-provided question-bank JSON. The original PDF and JSON ZIP are included under `dataset/sources/`, and the extracted source is stored in `dataset/question_bank_original.json`. The public problem titles and interface fields were retained. Each task also states assumptions about boundary conditions and scan timing so that its expected behavior can be tested consistently. The `source` and `adaptation_notes` fields identify the origin and any adaptation for each problem.

The 10 self-developed problems are original synthetic tasks; they do not combine or reproduce source problem statements. They cover pump rotation, conveyor jam reversal, tank-level control, batch mixing, staged fans, lead-lag compressors, safety-door trips, product-ejection pulses, dual-tank transfer, and heater diagnostics. Each task defines I/O, state boundaries, timing conditions, priorities, or reset behavior. We reviewed the safe state, threshold boundaries, and consistency between the requirements and LTL specifications.

Source	Easy	Medium	Hard	Total
Assigned public question bank	6	8	6	20
Original synthetic tasks	4	4	2	10
Total	10	12	8	30

Difficulty labels reflect control-state and logic complexity rather than code length. Easy tasks focus on one state, interlocks, hysteresis, or timing. Medium tasks add multi-step sequences, mode changes, fault recovery, or numeric boundaries. Hard tasks involve queues, dynamic selection, staged automation, lead-lag

equipment, or invariants spanning several states. The 30 tasks contain 279 typed and directed interface fields, 206 LTL specifications, and 112 test scenarios. Each task has at least three specifications and three test scenarios. After an interface audit, all 10 synthetic tasks had machine-readable I/O definitions. The canonical synthetic-task source is `dataset/synthetic_tasks.json`.

3.2 Scan-level LTL semantics

Each trace position pairs the sampled inputs with the state and outputs produced by that scan. A `prev` value comes from the preceding scan. `G p` means that `p` holds at every position, `X p` refers to the next position, and `F p` refers to a future position. Dataset v1.1 compares P04 and P12 accumulators with their preceding values and preserves them on stopped scans. Mode and stage guards keep automatic constraints out of manual control. Emergency-stop and reset priority are explicit in the revised properties. The archived outputs were generated before these edits.

This representation focuses on causality and safety at scan boundaries. Atoms such as `timerDone` must be interpreted by a target runtime model. Named macros such as `AtMostNOf(...)` and `validIPv4` must be expanded into syntax supported by the selected verifier adapter. PLCVerif was not available on the host, so the LTL specifications were not parsed or checked by a formal tool. Reviewable specification text is not evidence that a model checker proved the property.

4 Custom Agent Design

The agent uses the task JSON as its single source of truth. It combines the natural-language requirements, interface, assumptions, LTL, and test scenarios into one model input. Each task is handled with its own full context; state is not shared between tasks. This reduces cross-task contamination, but long-context compression has not been implemented. The model adapter calls DMXAPI through the OpenAI-compatible Chat Completions API, extracts one ST program from the response, and saves it.



The custom harness supplies the task statement, interface and assumptions. The current adapter imports the complete `PROGRAM`, `FUNCTION_BLOCK`, and `FUNCTION` units and schedules a driver that calls each one. It provides concrete variables for reference arguments and arrays. CODESYS must produce a nonempty offline boot application and a compiled signature for each candidate and the driver. The harness permits up to two repair attempts after source or compile errors.

The static precheck only looks for a program-unit declaration, closing markers, `VAR`/`END_VAR`, and a small set of control-structure terminators. It does not parse IEC syntax, check types, or compile for the target platform.

Actual compilation is performed by the adapter in `src/compiler.py`. Static checks are not treated as compile or runtime evidence; actual runtime results are reported separately.

The Pi strategy uses one generation with tools disabled. The custom strategy allows compiler feedback and up to two repairs. Both use the same task inputs and model endpoint within each model condition. The comparison measures these specific strategies with unequal tool and repair budgets. A framework-level comparison would require matched tools and budgets.

5 Experimental Design and Metrics

The four conditions are Pi + GPT, Pi + DeepSeek, custom harness + GPT, and custom harness + DeepSeek. Each condition covers all 30 tasks, for 120 condition-task cells. Temperature is fixed at 0, and the model IDs, task text, assumptions, LTL, and system prompt are aligned. Task order is shuffled with a fixed random seed. The formal custom-harness conditions allow up to two repair attempts.

The reported metrics are:

1. **First-pass compile rate:** number of tasks passing the first CODESYS compile divided by the total tasks in the condition. A task without non-empty ST counts as an end-to-end failure. The report also gives the rate among submitted source files to distinguish generation failures from compile failures.
2. **Post-repair compile rate:** number of tasks whose final CODESYS compile passed divided by the total tasks in the condition. The report also gives the rate among submitted source files and the number of repair attempts.
3. **Test pass rate:** passed test scenarios divided by scenarios actually executed. The report also gives the proportion of task outputs that passed every scenario in their suite.
4. **Formal checks:** each task is classified as PASS, FAIL, TIMEOUT, or NOT_RUN. Unexecuted checks are not counted as passes.
5. **Resource use:** model tokens, provider-reported cost, end-to-end latency, and repair attempts.

6 Experimental Results

6.1 Coverage

The formal matrix contains 30 tasks × 2 models × 2 agent conditions, for 120 cells. All 120 cells have results, and each task has a result in all four conditions. No cell was left unstarted because of the budget.

Task source	Fully paired	Partially run	Not started	Executed cells / planned cells
Assigned public question bank	20/20	0	0	80/80
Original synthetic tasks	10/10	0	0	40/40
Total	30/30	0	0	120/120

6.2 Code Generation and Linked Compile Results

In the initial DeepSeek matrix, no non-empty ST was extracted from any of the 60 cells. The diagnostic calls returned HTTP 200, but each completion reached the 768-token limit. After disabling the DeepSeek thinking mode using a supported API parameter, small requests began returning extractable `content`; the full recovery run produced non-empty ST in all 60 cells. This evidence supports the explanation that reasoning output consumed the generation budget and left no extractable body. Because the cost proxy deliberately did not store response bodies, the original responses cannot be reviewed cell by cell. Model settings and safe diagnostic metadata are in `experiments/config.example.json`, `src/adapters.py`, and `experiments/diagnostics/deepseek/`.

Four CODESYS compile calls timed out during the matrix. After the model requests had finished, the same CODESYS profile was used for separate retries. Three passed. The custom-harness P11 output still failed under the original adapter with an undeclared `R_TRIG` and incomplete control structure. The retry evidence is in `experiments/codesys_validation/deepseek_timeout_retries/` and does not overwrite the initial timeout records.

The historical matrix recorded 77 first-build passes, 39 failures and four timeouts. It recorded 90 final passes after retries. A later build-only check recorded 30 passes for the previously failing files. Both methods could accept an application without establishing that candidate code entered code generation, so the linked recheck below is the current offline measure.

Strategy and model	Linked pass	CODESYS error	Incomplete source
Pi and GPT-6.1 Sol	18	0	12
Custom harness and GPT-6.1 Sol	16	0	14
Pi and DeepSeek V4.1 Flash	9	2	19
Custom harness and DeepSeek V4.1 Flash	9	1	20
Total	52	3	65

All 120 source hashes were checked. The adapter rejected 55 files without a matching POU end marker and 10 with an unclosed block comment. The remaining 55 reached CODESYS and 52 passed. Three produced actual compiler diagnostics. A valid function block generated a boot application and a version with an undeclared identifier failed. Of the 90 historical passes, 44 did not pass under the corrected method. Six historical failures passed. The complete record, including the scheduled calls and candidate signatures, is in `experiments/linked_compile_review/`.

6.3 Cost, resource use, and unexecuted verification

Costs and model usage

Resource metric	Observed value
Formal matrix API usage	CNY 5.880938
Total project API usage, including pilot and diagnostics	CNY 6.099742 / CNY 18 limit
GPT-6.1 Sol project cost	CNY 5.258528
DeepSeek V4.1 Flash project cost	CNY 0.841214
API usage records and status	234 records: 233 HTTP 200, 1 budget rejection
Input / output tokens	435,406 / 171,446
Median request latency	GPT: 12,443 ms; DeepSeek: 6,979 ms

Compile, test, and verification coverage

Validation metric	Observed value
Uniform linked offline recheck	52/120 passed (43.3%). Three compiler errors and 65 incomplete sources
Source provenance	All 120 hashes match the archived final outputs. No model requests
Custom-agent CODESYS runtime plan	224 model-scenario executions (112 × 2 models); 88 run and 136 blocked
Custom-agent runtime results for executed scenarios	82 PASS, 6 FAIL; 56/112 distinct scenario IDs covered; all scenarios passed for 20/60 outputs; 3 outputs had assertion failures

Validation metric	Observed value
Runtime blockers	12 outputs skipped after formal offline compile failures; 25 outputs marked offline COMPILE_PASS failed to compile in the integrated test project
Additional STruC++ functional tests	20/20 PASS across four P02 outputs
CBMC bounded properties	918 checked across four translated P02 outputs; 0 failures
PLCVerif / full-dataset formal verification	PLCVerif was not installed; no full-dataset verification was run

The formal matrix used CNY 5.880938. Total recorded usage, including pilot runs, DeepSeek diagnostics, and recovery, was CNY 6.099742, below the CNY 18 limit. The runtime batch made no model API calls. It executed only a subset of the custom-agent outputs; the remaining scenarios were blocked by formal offline compile failures or by compile failures in the integrated test project. The three outputs with executed runtime assertion failures were P09 GPT, P14 GPT, and P16 GPT. They exposed a missing moving-average implementation, incorrect IPv4 text validation, and missing station-state transitions, respectively. The initial P06 timing assertions were revised to check below five seconds, reset, and then measure 5.25 seconds; both models passed 3/3. The initial S10 stateful case order was also revised; GPT passed 5/5 on rerun. The STruC++ and CBMC bonus results cover P02 only. PLCVerif was not run, so these results do not establish runtime correctness for all 30 tasks or satisfaction of every LTL property.

Cost and token figures come from `experiments/*usage*.jsonl`. The proxy records do not contain run IDs, and some row-level CSV costs may be missing or delayed. We therefore do not use the CSV to compare per-task costs. The proxy does not save prompts or response bodies; the original empty DeepSeek responses cannot be reviewed individually. The root-cause explanation is based on HTTP status, completion token counts, and controlled comparisons of the thinking-mode parameter.

6.4 Automated testing and bounded formal checks

To add automated testing and formal-check evidence, we selected P02 because it includes boundary arithmetic and state updates. The same scan-by-scan assertions were run against the four formal outputs: Pi and custom harness with GPT and DeepSeek. Each output ran five tests: the three P02 dataset scenarios and boundary cases for all 32 bits rising and all 32 bits falling. STruC++ 0.7.0 compiled and ran the tests, with 20/20 passes in total. The Pi DeepSeek output used an equivalent expression normalization to support STruC++ 0.7.0. The original ST was unchanged; the transformation and source hashes are recorded in the result JSON.

CBMC 5.95.1 then checked the STruC++-generated code for the four P02 outputs. The harness checks two arbitrary consecutive 32-bit input changes and properties for edge masks, changed/edge flags, counts, and previous-value updates, along with loop unwinding, bounds, pointers, and undefined shifts. All 918 checks passed, with no failures. This result applies only to the STruC++ translations of P02 and the listed bounded states. It does not prove equivalence to CODESYS runtime or to every PLC environment. Reproduction scripts, test templates, CBMC harnesses, logs, hashes, and per-output results are in `tests/structured_text/`, `tools/run_st_tests.py`, `tools/run_p02_cbmc.py`, `experiments/functional_tests/`, and `experiments/formal_verification/`.

7 Validity Limits and Further Work

The linked recheck measures whether the archived final sources can build in an offline CODESYS application. It does not establish functional behavior, exact interface conformance or PLC runtime correctness. GPT produced 18/30 passes with the Pi strategy and 16/30 with the custom strategy. Both DeepSeek strategies produced 9/30. These counts describe this experiment and do not isolate the effect of the frameworks. The previous 88/224 runtime executions used their original candidate selection and test vectors. The revised vectors have yet to be run on Control Win. The 918 CBMC checks apply to P02 translations, and the complete LTL set has not been checked by PLCVerif.

Further work can proceed in this order:

1. Repair the generated-code defects observed in P09, P14, and P16, then evaluate those repairs separately from the original matrix. Analyze the 25 outputs that failed to compile in the integrated runtime test project; fix runner or compatibility issues and rerun the blocked scenarios.
2. Install PLCVerif and configure an adapter. Expand named LTL macros into the tool's supported syntax, then check the state and timer semantics before verifying tasks.
3. Extend STruC++ functional tests to more tasks and add more state orderings, timing boundaries, and invalid-input cases.
4. Assess full-dataset correctness after scenario coverage is complete and blockers are resolved. Use more task types and model versions before drawing broader conclusions.

The original run capped model output at 768 tokens per request. The prevalence of incomplete archived outputs warrants measuring truncation explicitly in any future run. Total recorded project usage remains CNY 6.099742 out of CNY 18. The uniform compile recheck reused the archived sources and incurred no API costs.

Appendix A Project Artifacts

The 30 tasks, 279 interface fields, 206 formal specifications, and 112 test scenarios are in `dataset/tasks.json`. The experimental protocol is in `experiments/protocol.md`. The original 120-cell matrix and summary are in `experiments/results.csv` and `experiments/results_summary.json`. Current per-candidate compile evidence is in `experiments/linked_compile_review/`, and reusable DUT definitions are in `st/support_types/`. CODESYS timeout retries, the P02 runtime smoke test, and the custom-agent runtime scenario results are in `experiments/codesys_validation/`. Per-output STruC++ and CBMC results, source hashes, test templates, transformation evidence, and logs are in `experiments/functional_tests/`, `experiments/formal_verification/`, and `tests/structured_text/`. Generated ST files are in `st/runs/`. Usage records contain model, token, cost, latency, HTTP status, and timestamp, but no API key, prompt, or response body.

References

1. Course assignment and public question bank: `dataset/sources/作业说明中文原件.docx`, `dataset/sources/题库原始.pdf`, and `dataset/sources/题库原始 JSON.zip`.
2. DMXAPI OpenAI-compatible Chat Completions and cost fields: https://doc.dmxapi.cn/openai-chat.html and https://doc.dmxapi.cn/dmxapi-cost.html.
3. Pi coding-agent custom OpenAI-compatible provider and CLI documentation: https://pi.dev/docs/latest/custom-provider and https://pi.dev/docs/latest/cli.
4. CODESYS command-line and ScriptEngine documentation: https://content.helpme-codesys.com/en/CODESYS%20Development%20System/_cds_commandline.html, https://content.helpme-codesys.com/en/ScriptingEngine/ScriptIecLanguageObjectContainer.html, and https://content.helpme-codesys.com/en/ScriptingEngine/ScriptApplication.html.
5. DeepSeek Thinking Mode parameters and reasoning/content response fields: https://api-docs.deepseek.com/guides/thinking_mode/.
6. STruC++ automated testing guide: https://github.com/Autonomy-Logic/STruCpp/blob/development/docs/TESTING.md; CBMC bounded model checker: https://github.com/diffblue/cbmc.